

Chapter 21: Generics in Java

Personal Notes

In the Collections chapter you used types like `List<String>`, `HashMap<String, Integer>`, and `PriorityQueue<Integer>` everywhere. Those angle brackets are generics — and now it's time to understand how they actually work.

Generics give you ONE class or method that works safely with MANY types. Without them, you'd either copy-paste a class for every type, or give up on type safety entirely. Once you understand generics, you can read advanced Java code without flinching.

1. What are Generics?

Generics let you write CLASSES, INTERFACES, and METHODS that work with any type, while still keeping FULL type safety. The type is left as a placeholder (like `T` or `E`), and gets filled in when you actually use the class.

Think of it like a TEMPLATE — "this is a Box that holds something — you decide what."

Without Generics	With Generics
<pre>class IntegerBox { ... } class StringBox { ... } class DoubleBox { ... } class CarBox { ... }</pre>	<pre>class Box<T> { ... } Box<Integer> b1 = new Box<>(); Box<String> b2 = new Box<>(); Box<Double> b3 = new Box<>();</pre>
(one class per type)	(one class, any type)

2. Why We Need Generics

Before generics (Java 1.4 and earlier), collections held any object you put into them. That sounded flexible — but it was a disaster.

2.1 The Bad Old Days

```
// Before generics – using raw types
List names = new ArrayList();
names.add("Aman");
names.add(25); // oops – wrong type, but allowed!

String name = (String) names.get(0); // OK – first item is a String
String age = (String) names.get(1); // ClassCastException at runtime!
```

Two problems with this approach:

- No way to stop the wrong type from being added
- You needed to manually cast every time you took something out
- Errors only showed up at RUNTIME, not at compile time

2.2 With Generics

```
// Modern Java – using generics
List<String> names = new ArrayList<>();
names.add("Aman");
names.add(25);                // COMPILE ERROR – caught immediately

String name = names.get(0); // no cast needed – Java knows it's a String
```

Three big wins:

- Type safety — the compiler catches type errors BEFORE the program runs
- No casting needed — the compiler already knows the type
- Cleaner, self-documenting code — `List<String>` tells you exactly what's inside

Key insight: Generics let you push errors from runtime back to compile time. A runtime error crashes a user's app. A compile error never ships. That difference alone is reason enough to always use generics.

3. The Type Parameter

The thing in angle brackets like `<T>` is called a TYPE PARAMETER. It is a placeholder that stands in for a real type.

By convention, Java uses these single-letter names:

Letter	Stands For	Used When
T	Type	Generic placeholder for any type
E	Element	Used for elements in a collection (<code>List<E></code>)
K	Key	Used for keys in a map (<code>Map<K, V></code>)
V	Value	Used for values in a map (<code>Map<K, V></code>)
N	Number	Used when the type is some kind of number
R	Return	Sometimes used for return types in functional code

These are just conventions: Technically you could use any letter — even full words. But every Java developer expects T, E, K, V. Stick to the convention and your code will read naturally to anyone.

4. Generic Classes

A generic class is a class with one or more type parameters. The type stays abstract until someone actually creates an object.

4.1 A Simple Generic Box

```
public class Box<T> {
    private T value;

    public void set(T value) {
        this.value = value;
    }

    public T get() {
        return value;
    }
}

public class Main {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>();
        intBox.set(100);
        System.out.println(intBox.get());    // 100

        Box<String> strBox = new Box<>();
        strBox.set("Hello");
        System.out.println(strBox.get());    // Hello

        // intBox.set("oops");    // COMPILER ERROR – T is Integer here
    }
}
```

The same Box class works for any type. Each time you create a Box, you pick what T will be — and the compiler enforces it from that point on.

4.2 The Diamond Operator <>

Notice we wrote `new Box<>()` — empty angle brackets. This is the DIAMOND OPERATOR, added in Java 7. The compiler infers the type from the left side.

```
Box<Integer> b = new Box<Integer>();    // verbose – pre-Java 7 style
Box<Integer> b = new Box<>();           // cleaner – diamond inference
```

4.3 Multiple Type Parameters

A class can have more than one type parameter. The classic example is a Pair class.

```
public class Pair<K, V> {
```

```

private K key;
private V value;

public Pair(K key, V value) {
    this.key = key;
    this.value = value;
}

public K getKey() { return key; }
public V getValue() { return value; }
}

// Usage:
Pair<String, Integer> p1 = new Pair<>("Aman", 92);
Pair<Integer, String> p2 = new Pair<>(101, "Karan");

System.out.println(p1.getKey() + " → " + p1.getValue()); // Aman → 92

```

5. Generic Methods

Sometimes you don't need a whole generic class — just one method that works with any type. That's a **GENERIC METHOD**. The type parameter is declared right before the return type.

Syntax

```

public static <T> void printArray(T[] arr) {
    for (T element : arr) {
        System.out.println(element);
    }
}

```

The `<T>` before the return type declares `T` as a type parameter for this method. You can then use `T` anywhere in the method.

Full Example

```

public class Util {

    // Generic method – works with any type
    public static <T> void printAll(T[] arr) {
        for (T element : arr) {
            System.out.println(element);
        }
    }

    // Generic method that returns a value
    public static <T> T firstOf(T[] arr) {
        return arr[0];
    }
}

```

```

public class Main {
    public static void main(String[] args) {
        Integer[] nums = {1, 2, 3};
        String[] words = {"Java", "DSA", "OOP"};

        Util.printAll(nums);    // 1 2 3
        Util.printAll(words);   // Java DSA OOP

        Integer firstNum = Util.firstOf(nums);    // 1
        String firstWord = Util.firstOf(words);   // Java
    }
}

```

Generic class vs generic method: A generic class declares T once and uses it for all members. A generic method declares its own T just for that one method. Use a generic class when many methods share the same type. Use a generic method when only one operation needs it.

6. Bounded Type Parameters

Sometimes T "any type" is too loose. You want T to be specifically a Number, or something that implements Comparable. That's where bounded type parameters come in.

6.1 The extends Keyword

Use <T extends SomeClass> to say "T can be SomeClass or any subclass of it."

```

// T must be a Number (or a subclass like Integer, Double, etc.)
public static <T extends Number> double sum(List<T> list) {
    double total = 0;
    for (T num : list) {
        total += num.doubleValue();    // OK – Number has doubleValue()
    }
    return total;
}

// Usage:
List<Integer> ints = Arrays.asList(1, 2, 3);
List<Double> dbls = Arrays.asList(1.5, 2.5, 3.5);

sum(ints);    // 6.0
sum(dbls);    // 7.5
// sum(Arrays.asList("a","b"));    // COMPILE ERROR – String is not a Number

```

Why this is useful: Without the bound, T could be anything — including types that don't have a doubleValue() method. With <T extends Number>, the compiler GUARANTEES that whatever T is, it has Number's methods available.

6.2 Multiple Bounds

You can require T to satisfy multiple bounds using the & operator.

```
// T must be both a Number AND implement Comparable
public static <T extends Number & Comparable<T>> T max(List<T> list) {
    T max = list.get(0);
    for (T item : list) {
        if (item.compareTo(max) > 0) {
            max = item;
        }
    }
    return max;
}
```

Rule: When using multiple bounds, the CLASS must come first, followed by interfaces. <T extends Number & Comparable<T>> is correct. You can have only one class bound, but multiple interface bounds.

7. Wildcards: ?

The question mark ? in generics is called a WILDCARD. It means "some unknown type — I don't care which." Used mostly in method parameters when you only need to READ from a generic collection.

7.1 The Unbounded Wildcard: <?>

```
public static void printList(List<?> list) {
    for (Object item : list) {
        System.out.println(item);
    }
}

// Works with ANY list:
printList(Arrays.asList(1, 2, 3));
printList(Arrays.asList("a", "b", "c"));
printList(Arrays.asList(true, false));
```

List<?> means "a list of something — I don't know what." You can read items as Objects, but you can't add anything (except null) because the compiler doesn't know what type would be safe.

7.2 Upper Bounded: <? extends Type>

"Some type that is Type or a subclass of Type." Used when you want to READ from the collection.

```
public static double sumOfList(List<? extends Number> list) {
    double total = 0;
    for (Number n : list) {
        total += n.doubleValue();
    }
}
```

```

    return total;
}

sumOfList(Arrays.asList(1, 2, 3));           // OK – List<Integer>
sumOfList(Arrays.asList(1.5, 2.5));         // OK – List<Double>
// sumOfList(Arrays.asList("a", "b"));      // ERROR – String not a Number

```

7.3 Lower Bounded: <? super Type>

"Some type that is Type or a SUPERCLASS of Type." Used when you want to WRITE to the collection.

```

public static void addNumbers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
    list.add(3);
}

List<Integer> ints = new ArrayList<>();
List<Number>  nums = new ArrayList<>();
List<Object>  objs = new ArrayList<>();

addNumbers(ints);    // OK – Integer is Integer
addNumbers(nums);    // OK – Number is a parent of Integer
addNumbers(objs);    // OK – Object is the parent of everything

```

7.4 The PECS Rule

PECS is the famous mnemonic for when to use each wildcard:

```

P   E   C   S
|   |   |   |
Producer → use extends (you READ from it)
      |
      Extends
          Consumer → use super (you WRITE to it)
                |
                Super

```

In plain English:

- If a collection PRODUCES values for you (you READ from it), use <? extends T>
- If a collection CONSUMES values from you (you WRITE to it), use <? super T>
- If you do both, use a concrete type like List<T>

8. Generic Interfaces

Interfaces can be generic too. You've already used them — Comparable<T>, Iterable<T>, List<E>, Map<K,V> are all generic interfaces.

Defining a Generic Interface

```
public interface Container<T> {
    void add(T item);
    T get(int index);
    int size();
}

// A class can implement a generic interface in two ways:

// 1. Specify the type
public class StringContainer implements Container<String> {
    private List<String> items = new ArrayList<>();

    public void add(String item) { items.add(item); }
    public String get(int i)      { return items.get(i); }
    public int size()             { return items.size(); }
}

// 2. Stay generic
public class GenericContainer<T> implements Container<T> {
    private List<T> items = new ArrayList<>();

    public void add(T item) { items.add(item); }
    public T get(int i)     { return items.get(i); }
    public int size()       { return items.size(); }
}
```

9. Type Erasure (Behind the Scenes)

Java's generics work through a process called TYPE ERASURE. After your code compiles, the type parameters are ERASED — replaced with `Object` (or with their bound, if specified). The runtime never knows the generic type.

What you write	What the compiler produces
<pre>public class Box<T> { T value; T get() void set(T v) }</pre>	<pre>public class Box { Object value; Object get() void set(Object v) }</pre>

Why it's done this way: backward compatibility. Java had to keep old non-generic code working when generics were added in Java 5. So generics work at compile-time only — the compiled bytecode looks essentially the same as pre-generics code.

Consequences of type erasure: You CANNOT do things like `new T()` (no constructor info), `T.class` (no class info), or `instanceof T` at runtime. You also cannot have `List<Integer>` and `List<String>` as separate overloads — at runtime they're both just `List`.

10. Generic Class vs Generic Method — Quick Comparison

Aspect	Generic Class	Generic Method
Where T is declared	On the class itself	On the method itself
Syntax	class Box<T> { ... }	<T> void method(T arg) { ... }
Scope of T	All methods in the class	Just that one method
Best for	When many methods share a type	When only one operation is generic
Example	Container<String>, ArrayList<Integer>	Collections.sort(list)

11. Common Mistakes

Mistake 1: Using raw types

```
List list = new ArrayList();           // raw – no type safety
List<String> list = new ArrayList<>();  // safe – generic
```

Raw types lose all type safety. The compiler can't help you anymore. Always use generics — even if you write `Object` inside the brackets.

Mistake 2: Trying to use primitives directly

```
List<int> nums = new ArrayList<>();      // ERROR – primitives not allowed
List<Integer> nums = new ArrayList<>();  // use wrapper type instead
```

Generics only work with REFERENCE types. For primitives, use their wrapper classes: `int` → `Integer`, `double` → `Double`, `char` → `Character`, `boolean` → `Boolean`. Autoboxing converts back and forth automatically.

Mistake 3: Forgetting the diamond

```
// Verbose:
Map<String, List<Integer>> map = new HashMap<String, List<Integer>>();

// Cleaner:
Map<String, List<Integer>> map = new HashMap<>();
```

Mistake 4: Creating arrays of generic types

```
List<String>[] arr = new List<String>[10];  // ERROR – not allowed

// Use a List of Lists instead:
List<List<String>> arr = new ArrayList<>();
```

Java doesn't allow generic arrays because of type erasure issues. Use a List of Lists, or an `ArrayList<T>`.

Mistake 5: Confusing <? extends T> and <? super T>

Remember PECS: extends for reading (Producer Extends), super for writing (Consumer Super). Mix them up and the compiler will fight you.

Mistake 6: Trying to instantiate the type parameter

```
public class Box<T> {  
    T value = new T();    // ERROR – type T is erased at runtime  
}
```

Because of type erasure, you can't create new T(). Workarounds exist (passing a Class<T> reference), but they're advanced.

12. Best Practices

- ALWAYS use generics — never use raw types unless dealing with very old code
- Use the diamond operator <> on the right side — it's cleaner
- Use the standard convention: T, E, K, V — other Java devs will understand instantly
- Use bounded type parameters (<T extends Number>) when you need methods from a specific type
- Follow PECS for wildcards: Producer Extends, Consumer Super
- Prefer generic methods over wildcards when you need to refer to the same type in multiple places
- Use meaningful names for type parameters in complex APIs (TElement, TKey) for readability

13. Quick Reference

Syntax / Concept	Meaning
<T>	Type parameter — placeholder for any type
<E>	Element — used for collection elements
<K, V>	Key and Value — used for maps
<T extends C>	T must be C or a subclass of C
<T extends A & B>	T must satisfy multiple bounds
<?>	Unbounded wildcard — "some unknown type"
<? extends T>	Upper bounded — "T or any subclass" (for reading)
<? super T>	Lower bounded — "T or any superclass" (for writing)
new Box<>()	Diamond operator — type inferred from left side

Syntax / Concept	Meaning
PECS	Producer Extends, Consumer Super
Type Erasure	Generic info is removed at runtime

14. Practice Problems

Try these to lock in generics. Each one builds a different aspect of the feature.

1. Create a generic class `Box<T>` with `set()` and `get()` methods. Use it with `Integer`, `String`, and `Double`.
2. Create a generic `Pair<K, V>` class with a constructor and getters. Print a `Pair<String, Integer>`.
3. Write a generic method `swap(T[] arr, int i, int j)` that swaps two elements in any array.
4. Write a generic method `printArray(T[] arr)` that prints elements of any array type.
5. Write a method `<T extends Number> double averageOf(List<T> list)` that works on any numeric list.
6. Build a generic `Stack<T>` class using `ArrayList` internally. Support `push()`, `pop()`, and `peek()`.
7. Build a generic `LinkedList<T>` with `addFirst()`, `addLast()`, and `removeFirst()` methods.
8. Write a method `<T extends Comparable<T>> T findMax(List<T> list)` that returns the largest element.
9. Create a generic interface `Container<T>` with `add()` and `get()`. Implement it as `StringContainer` and `IntegerContainer`.
10. Write a method that takes a `List<? extends Number>` and returns the sum as a double.
11. Demonstrate PECS by writing two methods: one that copies from a `List<? extends T>` source to a `List<? super T>` destination.
12. Build a generic `KeyValueStore<K, V>` with `put()`, `get()`, and `remove()` backed by a `HashMap`.

15. Final Takeaway

Generics are the feature that makes the Collections Framework usable. They turn type errors from runtime bugs into compile errors, eliminate manual casting, and let you write one class that works safely with any type.

Most of the time you'll just USE generics from the standard library: `List<String>`, `Map<Integer, User>`, `Optional<T>`. Occasionally you'll write your own generic classes or methods. Either way, understanding the syntax — type parameters, bounds, wildcards — is the difference between reading Java code fluently and getting stuck on every angle bracket.

Key Takeaway: Generics push errors from runtime to compile time. That single shift is worth the entire complexity of `T`, `E`, `K`, `V`, `?`, `extends`, and `super`. The goal isn't to memorize syntax — it's to write code where the compiler catches bugs BEFORE your users do.

16. What's Next?

Now that generics are clear, the natural next Java topics are:

- Comparable & Comparator — generic interfaces for custom sorting
- Exception Handling — try, catch, finally, custom exceptions
- Lambda Expressions & Functional Interfaces (Java 8+)
- Java Streams API — generics + lambdas combined into one-line data processing
- Optional<T> — safely handling "maybe null" values
- Generic Method References and BiFunction / Function interfaces